

第四單元：類別與物件導向程式設計 - 詳細教學內容

單元學習目標

- 理解物件導向程式設計（OOP）的核心思想
- 掌握類別（Class）的定義與設計原則
- 學會建立物件實例並操作其屬性與方法
- 理解封裝（Encapsulation）、繼承（Inheritance）、多型（Polymorphism）的概念
- 能夠設計並實作物件之間的互動

4.1 物件導向程式設計基礎

什麼是物件導向程式設計？

物件導向程式設計（Object-Oriented Programming, OOP）是一種以「物件」為核心的程式設計思維，模擬真實世界中的事物與行為。

生活比喻：

現實世界		Java 程式
汽車的設計圖	→	類別（Class）
一台實際的車	→	物件（Object）
顏色、型號	→	屬性（Attribute / Field）
發動、煞車	→	方法（Method）

OOP 三大核心特性

特性	說明	生活比喻
封裝 (Encapsulation)	將資料與操作包裝在一起，隱藏內部細節	電視遙控器：按鈕操作，不需知道內部電路
繼承 (Inheritance)	子類別繼承父類別的屬性與方法	電動車繼承了汽車的特性，並加上電池功能
多型 (Polymorphism)	同一方法在不同物件有不同行為	同樣是「叫聲」，狗吠貓喵，各有不同

類別 vs 物件

```
// 類別是「設計圖」，定義了物件的結構
public class Dog {
    String name;
```

```
String breed;
int age;

void bark() {
    System.out.println(name + " 說: 汪汪! ");
}

}

public class Main {
    public static void main(String[] args) {
        // 物件是根據設計圖「建造出來的實體」
        Dog myDog = new Dog(); // 建立物件
        myDog.name = "小黑";
        myDog.breed = "拉不拉多";
        myDog.age = 3;
        myDog.bark(); // 輸出: 小黑 說: 汪汪!

        Dog friendDog = new Dog(); // 同一類別, 不同物件
        friendDog.name = "小白";
        friendDog.bark(); // 輸出: 小白 說: 汪汪!
    }
}
```

4.2 類別設計與實作

類別的基本結構

```
public class ClassName {
    // ① 欄位 (屬性): 描述物件的狀態
    private dataType fieldName;

    // ② 建構子: 初始化物件
    public ClassName(dataType param) {
        this.fieldName = param;
    }

    // ③ 方法: 描述物件的行為
    public returnType methodName() {
        // 方法內容
    }

    // ④ Getter / Setter: 存取私有屬性
    public dataType getFieldName() {
        return fieldName;
    }

    public void setFieldName(dataType value) {
        this.fieldName = value;
    }
}
```

```
}  
}
```

實作範例：學生類別

```
public class Student {  
    // ① 欄位 (私有, 受封裝保護)  
    private String name;  
    private int grade;  
    private double gpa;  
  
    // ② 建構子: 帶參數, 初始化物件  
    public Student(String name, int grade, double gpa) {  
        this.name = name;  
        this.grade = grade;  
        this.gpa = gpa;  
    }  
  
    // ③ 無參數建構子 (預設建構子)  
    public Student() {  
        this.name = "未知";  
        this.grade = 10;  
        this.gpa = 0.0;  
    }  
  
    // ④ Getter 方法  
    public String getName() {  
        return name;  
    }  
  
    public int getGrade() {  
        return grade;  
    }  
  
    public double getGpa() {  
        return gpa;  
    }  
  
    // ⑤ Setter 方法 (含資料驗證)  
    public void setGpa(double gpa) {  
        if (gpa >= 0.0 && gpa <= 4.0) {  
            this.gpa = gpa;  
        } else {  
            System.out.println("GPA 必須介於 0.0 到 4.0 之間!");  
        }  
    }  
  
    // ⑥ 行為方法  
    public void study() {  
        System.out.println(name + " 正在認真讀書!");  
    }  
}
```

```
public boolean isHonorRoll() {
    return gpa >= 3.5;
}

// ⑦ toString 方法: 方便顯示物件資訊
public String toString() {
    return "學生: " + name + ", 年級: " + grade + ", GPA: " + gpa;
}
}
```

```
public class StudentDemo {
    public static void main(String[] args) {
        // 使用帶參數的建構子
        Student alice = new Student("Alice", 11, 3.8);
        Student bob = new Student("Bob", 10, 2.9);

        // 呼叫方法
        alice.study();
        System.out.println(alice); // 自動呼叫 toString()
        System.out.println("Alice 是榮譽榜嗎? " + alice.isHonorRoll());

        // 使用 Setter 修改屬性
        bob.setGpa(3.1);
        System.out.println(bob);

        // 嘗試設定無效的 GPA
        alice.setGpa(5.0); // 輸出錯誤提示
    }
}
```

執行結果:

```
Alice 正在認真讀書!
學生: Alice, 年級: 11, GPA: 3.8
Alice 是榮譽榜嗎? true
學生: Bob, 年級: 10, GPA: 3.1
GPA 必須介於 0.0 到 4.0 之間!
```

this 關鍵字

this 指向「當前物件本身」，常用於區分欄位名稱與參數名稱。

```
public class Circle {
    private double radius;

    // 若參數名稱與欄位名稱相同，必須用 this 區分
```

```
public Circle(double radius) {  
    this.radius = radius; // this.radius 是欄位, radius 是參數  
}  
  
public double getArea() {  
    return Math.PI * this.radius * this.radius;  
}  
}
```

4.3 封裝 (Encapsulation)

為什麼需要封裝?

封裝就像「膠囊」，將資料與相關方法包在一起，並限制外部直接存取內部資料。

問題範例 (無封裝)：

```
// ❌ 不好的設計：直接存取欄位，無法驗證資料  
public class BankAccount {  
    public double balance; // 外部可任意修改!  
}  
  
public class Main {  
    public static void main(String[] args) {  
        BankAccount account = new BankAccount();  
        account.balance = -9999; // ❌ 負數餘額，完全不合理!  
    }  
}
```

解決方案 (有封裝)：

```
// ✅ 良好的設計：使用封裝保護資料  
public class BankAccount {  
    private String owner;  
    private double balance;  
  
    public BankAccount(String owner, double initialBalance) {  
        this.owner = owner;  
        // 建構子也可加入驗證  
        this.balance = (initialBalance >= 0) ? initialBalance : 0;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0) {
```

```

        balance += amount;
        System.out.printf("存入 %.2f 元, 餘額: %.2f 元%n", amount, balance);
    } else {
        System.out.println("存款金額必須大於 0! ");
    }
}

public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        System.out.printf("提領 %.2f 元, 餘額: %.2f 元%n", amount, balance);
    } else if (amount > balance) {
        System.out.println("餘額不足! ");
    } else {
        System.out.println("提領金額必須大於 0! ");
    }
}

public String toString() {
    return owner + " 的帳戶, 餘額: " + balance + " 元";
}
}

```

```

public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("小明", 1000);
        System.out.println(account);

        account.deposit(500);
        account.withdraw(200);
        account.withdraw(2000); // 餘額不足

        // account.balance = -9999; // ❌ 編譯錯誤! 無法直接存取私有欄位
    }
}

```

執行結果:

```

小明 的帳戶, 餘額: 1000.0 元
存入 500.00 元, 餘額: 1500.00 元
提領 200.00 元, 餘額: 1300.00 元
餘額不足!

```

存取修飾詞

修飾詞	同一類別	同一套件	子類別	所有地方
private	✓	✗	✗	✗

修飾詞	同一類別	同一套件	子類別	所有地方
(預設)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

最佳實踐：欄位使用 `private`，方法使用 `public`。

♥ 4.4 物件之間的互動

物件作為參數傳遞

```
public class Rectangle {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double getArea() {
        return width * height;
    }

    public double getPerimeter() {
        return 2 * (width + height);
    }

    // 物件作為參數：比較兩個矩形的面積
    public boolean isLargerThan(Rectangle other) {
        return this.getArea() > other.getArea();
    }

    public String toString() {
        return String.format("矩形(%.1f x %.1f)", width, height);
    }
}
```

```
public class RectangleDemo {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(5, 3);
        Rectangle r2 = new Rectangle(4, 4);

        System.out.println(r1 + " 面積: " + r1.getArea());
        System.out.println(r2 + " 面積: " + r2.getArea());
    }
}
```

```
        if (r1.isLargerThan(r2)) {
            System.out.println(r1 + " 比較大");
        } else {
            System.out.println(r2 + " 比較大");
        }
    }
}
```

執行結果：

```
矩形(5.0 × 3.0) 面積: 15.0
矩形(4.0 × 4.0) 面積: 16.0
矩形(4.0 × 4.0) 比較大
```

物件的組合 (Composition)

一個類別的欄位可以是另一個類別的物件，這稱為「組合」 (Has-A 關係)。

```
public class Address {
    private String city;
    private String street;
    private String zipCode;

    public Address(String city, String street, String zipCode) {
        this.city = city;
        this.street = street;
        this.zipCode = zipCode;
    }

    public String toString() {
        return zipCode + " " + city + " " + street;
    }
}
```

```
public class Person {
    private String name;
    private int age;
    private Address address; // Person [擁有] Address 物件

    public Person(String name, int age, Address address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    public void introduce() {
        System.out.println("我是 " + name + ", " + age + " 歲");
    }
}
```



```
        System.out.println("住在: " + address);
    }
}
```

```
public class PersonDemo {
    public static void main(String[] args) {
        Address addr = new Address("台北市", "信義路一段", "110");
        Person person = new Person("陳小華", 17, addr);
        person.introduce();
    }
}
```

執行結果：

```
我是 陳小華, 17 歲
住在: 110 台北市 信義路一段
```

物件陣列

```
public class Classroom {
    public static void main(String[] args) {
        // 建立 Student 物件陣列
        Student[] students = new Student[3];
        students[0] = new Student("Alice", 11, 3.8);
        students[1] = new Student("Bob", 10, 3.2);
        students[2] = new Student("Carol", 11, 3.6);

        // 遍歷陣列並顯示資訊
        System.out.println("=== 班級名單 ===");
        for (Student s : students) {
            System.out.println(s);
            if (s.isHonorRoll()) {
                System.out.println("    ★ 榮譽榜學生");
            }
        }
    }
}
```

執行結果：

```
=== 班級名單 ===
學生: Alice, 年級: 11, GPA: 3.8
    ★ 榮譽榜學生
學生: Bob, 年級: 10, GPA: 3.2
```

學生: Carol, 年級: 11, GPA: 3.6

★ 榮譽榜學生

4.5 繼承 (Inheritance)

繼承的概念

繼承表示「是一種」 (Is-A) 的關係：子類別是父類別的特殊化版本，繼承父類別的屬性與方法，並可加入自己獨有的功能。

```
Animal (動物)
├─ Dog (狗) : 繼承 Animal, 加上 breed (品種)
├─ Cat (貓) : 繼承 Animal, 加上 indoor (是否室內貓)
└─ Bird (鳥) : 繼承 Animal, 加上 canFly (是否會飛)
```

extends 關鍵字

```
// 父類別 (Superclass / Parent Class)
public class Animal {
    protected String name;
    protected int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void eat() {
        System.out.println(name + " 正在吃東西");
    }

    public void sleep() {
        System.out.println(name + " 正在睡覺");
    }

    public String toString() {
        return "動物: " + name + " (" + age + " 歲) ";
    }
}
```

```
// 子類別 (Subclass / Child Class)
public class Dog extends Animal {
    private String breed;

    // super() 呼叫父類別的建構子
    public Dog(String name, int age, String breed) {
```

```

        super(name, age);        // 呼叫 Animal(name, age)
        this.breed = breed;
    }

    // 子類別特有的方法
    public void bark() {
        System.out.println(name + " 說: 汪汪! ");
    }

    public void fetch() {
        System.out.println(name + " 去撿球了! ");
    }

    // 覆寫 (Override) 父類別的方法
    @Override
    public String toString() {
        return "狗: " + name + " (品種: " + breed + ", " + age + " 歲) ";
    }
}

```

```

public class Cat extends Animal {
    private boolean isIndoor;

    public Cat(String name, int age, boolean isIndoor) {
        super(name, age);
        this.isIndoor = isIndoor;
    }

    public void meow() {
        System.out.println(name + " 說: 喵~");
    }

    @Override
    public void eat() {
        // 覆寫父類別方法, 加入自己的行為
        System.out.println(name + " 挑剔地吃著貓罐頭");
    }

    @Override
    public String toString() {
        String type = isIndoor ? "室內貓" : "室外貓";
        return "貓: " + name + " (" + type + ", " + age + " 歲) ";
    }
}

```

```

public class InheritanceDemo {
    public static void main(String[] args) {
        Dog dog = new Dog("小黑", 3, "拉不拉多");
        Cat cat = new Cat("咪咪", 5, true);
    }
}

```

```
// 使用繼承來的方法
dog.eat();
dog.sleep();

// 使用子類別特有的方法
dog.bark();
dog.fetch();

// Cat 覆寫了 eat()
cat.eat();
cat.meow();

System.out.println(dog);
System.out.println(cat);

// 多型：父類別型別可以指向子類別物件
Animal a1 = new Dog("阿旺", 2, "柴犬");
Animal a2 = new Cat("花花", 4, false);
a1.eat(); // 呼叫 Dog 的 eat() (繼承自 Animal)
a2.eat(); // 呼叫 Cat 覆寫的 eat()
    }
}
```

執行結果：

```
小黑 正在吃東西
小黑 正在睡覺
小黑 說：汪汪！
小黑 去撿球了！
咪咪 挑剔地吃著貓罐頭
咪咪 說：喵～
狗：小黑（品種：拉不拉多，3 歲）
貓：咪咪（室內貓，5 歲）
阿旺 正在吃東西
花花 挑剔地吃著貓罐頭
```

super 關鍵字

super 用來存取父類別的成員，常見用法：

```
public class Vehicle {
    protected String brand;
    protected int year;

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }
}
```

```
        public String describe() {
            return brand + " (" + year + " 年) ";
        }
    }

    public class ElectricCar extends Vehicle {
        private int batteryRange; // 電池續航里程 (km)

        public ElectricCar(String brand, int year, int batteryRange) {
            super(brand, year); // 呼叫父類別建構子
            this.batteryRange = batteryRange;
        }

        @Override
        public String describe() {
            // super.describe() 呼叫父類別的 describe()
            return super.describe() + ", 電動車, 續航: " + batteryRange + " km";
        }
    }

    public class SuperDemo {
        public static void main(String[] args) {
            ElectricCar tesla = new ElectricCar("Tesla Model 3", 2024, 560);
            System.out.println(tesla.describe());
        }
    }
}
```

執行結果：

```
Tesla Model 3 (2024 年) , 電動車, 續航: 560 km
```

@Override 方法覆寫

概念	說明
方法覆寫 (Override)	子類別重新定義父類別的方法
覆寫條件	方法名稱、參數列表、回傳型別必須相同
@Override 標記	讓編譯器確認是否正確覆寫，建議加上
存取修飾詞	覆寫後的方法不能比父類別更嚴格

4.6 綜合實作練習

練習一：設計圖書類別

```
public class Book {
    private String title;
    private String author;
    private double price;
    private boolean isAvailable;

    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
        this.isAvailable = true;
    }

    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public double getPrice() { return price; }
    public boolean isAvailable() { return isAvailable; }

    public boolean borrow() {
        if (isAvailable) {
            isAvailable = false;
            System.out.println("《" + title + "》已借出");
            return true;
        } else {
            System.out.println("《" + title + "》目前不可借閱");
            return false;
        }
    }

    public void returnBook() {
        isAvailable = true;
        System.out.println("《" + title + "》已歸還");
    }

    @Override
    public String toString() {
        String status = isAvailable ? "可借閱" : "已借出";
        return String.format("《%s》作者: %s, 售價: %.0f 元, 狀態: %s",
                               title, author, price, status);
    }
}
```

```
public class Library {
    public static void main(String[] args) {
        Book[] books = {
            new Book("Java 程式設計", "林小明", 580),
            new Book("資料結構與演算法", "陳大偉", 720),
            new Book("物件導向設計", "王美玲", 650)
        };
    }
}
```

```
        System.out.println("=== 圖書館藏書 ===");
        for (Book b : books) {
            System.out.println(b);
        }

        System.out.println("\n=== 借閱操作 ===");
        books[0].borrow();
        books[0].borrow(); // 再次嘗試借閱

        System.out.println("\n=== 歸還操作 ===");
        books[0].returnBook();
        books[0].borrow(); // 歸還後可再次借閱
    }
}
```

練習二：形狀繼承體系

```
// 抽象概念：形狀
public class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    public double getArea() {
        return 0; // 子類別應覆寫此方法
    }

    public String toString() {
        return color + " 形狀, 面積: " + String.format("%.2f", getArea());
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public String toString() {
        return color + " 圓形 (半徑 " + radius + ") , 面積: "
            + String.format("%.2f", getArea());
    }
}
```

```
}  
}  
  
public class Rectangle extends Shape {  
    private double width;  
    private double height;  
  
    public Rectangle(String color, double width, double height) {  
        super(color);  
        this.width = width;  
        this.height = height;  
    }  
  
    @Override  
    public double getArea() {  
        return width * height;  
    }  
  
    @Override  
    public String toString() {  
        return color + " 矩形 (" + width + " × " + height + ") , 面積: "  
            + String.format("%.2f", getArea());  
    }  
}  
  
public class ShapeDemo {  
    public static void main(String[] args) {  
        Shape[] shapes = {  
            new Circle("紅色", 5),  
            new Rectangle("藍色", 4, 6),  
            new Circle("綠色", 3),  
            new Rectangle("黃色", 8, 2)  
        };  
  
        System.out.println("=== 形狀清單 ===");  
        double totalArea = 0;  
        for (Shape s : shapes) {  
            System.out.println(s);  
            totalArea += s.getArea();  
        }  
        System.out.printf("%n所有形狀的總面積: %.2f%n", totalArea);  
    }  
}
```

執行結果：

```
=== 形狀清單 ===  
紅色 圓形 (半徑 5.0) , 面積: 78.54  
藍色 矩形 (4.0 × 6.0) , 面積: 24.00  
綠色 圓形 (半徑 3.0) , 面積: 28.27  
黃色 矩形 (8.0 × 2.0) , 面積: 16.00
```


所有形狀的總面積：146.81

 4.7 概念對照整理

類別結構速查表

```
public class MyClass {
    // 1. 欄位 (屬性)
    private int id;
    private String name;

    // 2. 建構子 (與類別同名, 無回傳型別)
    public MyClass(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // 3. Getter (取得私有欄位值)
    public int getId() { return id; }
    public String getName() { return name; }

    // 4. Setter (設定私有欄位值)
    public void setName(String name) { this.name = name; }

    // 5. 一般方法
    public void display() {
        System.out.println(id + ": " + name);
    }

    // 6. toString (文字表示)
    @Override
    public String toString() {
        return "MyClass{id=" + id + ", name=" + name + "}";
    }
}
```

繼承速查表

語法	說明	範例
extends	繼承父類別	class Dog extends Animal
super(...)	呼叫父類別建構子	super(name, age)
super.method()	呼叫父類別方法	super.describe()
@Override	覆寫父類別方法	標記在方法上方
protected	子類別可存取的欄位/方法	protected String name

常見錯誤與修正

常見錯誤	說明	修正方式
忘記呼叫 <code>super()</code>	子類別建構子未呼叫父類別建構子	在第一行加上 <code>super(...)</code>
直接存取 <code>private</code> 欄位	子類別無法存取父類別的 <code>private</code> 成員	改用 <code>protected</code> 或 Getter
覆寫時簽章不符	方法名稱或參數不一致導致重載而非覆寫	加上 <code>@Override</code> 讓編譯器檢查
<code>this</code> 與參數同名混淆	建構子內欄位與參數混淆	明確使用 <code>this.欄位名稱</code>

4.8 隨堂測驗

選擇題

1. 下列哪個關鍵字用於繼承父類別？

- A) `implements`
- B) `extends`
- C) `inherits`
- D) `super`

2. 在子類別建構子中，`super()` 必須放在哪裡？

- A) 建構子的最後一行
- B) 建構子的第一行
- C) 任意位置
- D) 不需要加入

3. 封裝的主要目的是什麼？

- A) 提升程式執行速度
- B) 減少程式碼行數
- C) 隱藏內部實作，控制存取權限
- D) 讓程式更容易除錯

4. 下列程式碼的輸出為何？

```
Animal a = new Dog("Buddy", 2, "Golden");
a.eat();
```

假設 `Dog` 繼承 `Animal`，且 `Dog` 沒有覆寫 `eat()`。

- A) 編譯錯誤
- B) 執行時錯誤
- C) 執行 `Dog` 的 `eat()`
- D) 執行 `Animal` 的 `eat()`

5. `@Override` 標記的作用是？

- A) 強制子類別必須覆寫方法
- B) 告訴編譯器檢查此方法是否正確覆寫父類別方法
- C) 讓方法執行效率提高
- D) 避免方法被其他類別呼叫

簡答題

1. 請說明 `private`、`protected`、`public` 三種存取修飾詞的差異。
2. 請說明 OOP 三大特性（封裝、繼承、多型）各自解決了什麼問題。
3. 在以下程式碼中找出錯誤並修正：

```
public class Animal {
    private String name;
    public Animal(String name) {
        this.name = name;
    }
}

public class Dog extends Animal {
    private String breed;
    public Dog(String name, String breed) {
        // 忘記呼叫父類別建構子
        this.breed = breed;
    }
    public void info() {
        System.out.println(name + " is a " + breed); // 存取 private 欄位
    }
}
```



隨堂測驗解答

選擇題： 1-B、2-B、3-C、4-D、5-B

簡答題 3 修正版本：

```
public class Animal {
    protected String name; // 改為 protected, 讓子類別可存取
    public Animal(String name) {
        this.name = name;
    }
}

public class Dog extends Animal {
    private String breed;
    public Dog(String name, String breed) {
        super(name); // ✓ 正確呼叫父類別建構子
        this.breed = breed;
    }
}
```

```
    }  
    public void info() {  
        System.out.println(name + " is a " + breed); // ✅ name 現在是 protected  
    }  
}
```

單元重點總結

OOP 核心概念

- ├─ 類別 (Class)
 - ├─ 欄位 (Fields): 物件的狀態/資料
 - ├─ 建構子 (Constructor): 初始化物件
 - ├─ 方法 (Methods): 物件的行為
 - └─ Getter / Setter: 封裝存取
- ├─ 封裝 (Encapsulation)
 - ├─ private 欄位
 - ├─ public 方法
 - └─ 資料驗證
- ├─ 繼承 (Inheritance)
 - ├─ extends 關鍵字
 - ├─ super() 建構子呼叫
 - ├─ @Override 方法覆寫
 - └─ protected 存取修飾詞
- └─ 物件互動
 - ├─ 物件作為參數
 - ├─ 組合 (Composition)
 - └─ 物件陣列

學習建議： OOP 概念需要大量練習才能真正掌握。建議從設計簡單的類別開始，逐步加入封裝與繼承，並嘗試思考「這個物件在現實世界中代表什麼？」